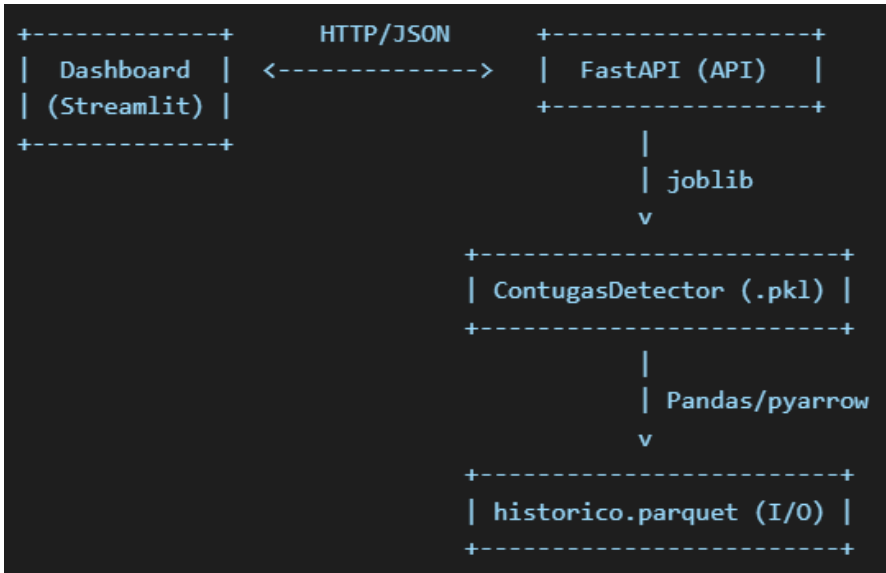


Manual Técnico

Arquitectura general



Componentes

Módulo	Descripción
contugas_detector.py	Carga <code>kmeans.pkl</code> , <code>scaler_kmeans.pkl</code> y un <i>Isolation Forest</i> por segmento. Expone <code>.predict()</code> thread-safe.
contugas_api.py	Endpoints REST: <code>/health</code> , <code>/predict</code> , <code>/batch_predict</code> , <code>/ingest</code> , <code>/plot/{cliente}</code> . Usa el detector y <code>oper. parquet</code> .
streamlit_app.py	UI de usuario: explora datos, lanza predicciones e ingesta. Llama a la API vía <code>requests</code> .
parquet_io.py	Utilidades I/O para construir o anexar registros a <code>historico.parquet</code> .
models.py	Ejecuta el modelo de segmentación de k-means y luego el <i>Isolation Forest</i> para luego guardarlos y que se disponibilicen para la API

Flujo de datos

1. **Ingesta:** `/ingest` escribe nuevas filas al Parquet (append).

2. **Predicción:** `/predict` → `ContugasDetector.predict()` → Isolation Forest → JSON.
3. **Dashboard** actualiza cache (`st.cache_data.clear()`) después de cada ingesta.

Entrenamiento / Retraining

Script `models.py` (en el repo) re-entrena K-Means + Isolation Forest con la versión local de Python y actualiza la carpeta `models/`. Programar (ej. Task Scheduler) cada mes:

```
python reexport_models.py && echo %date% >> retrain.log
```

Docker (opcional)

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "contugas_api:app", "--host", "0.0.0.0", "--port", "8000"]
```

Construir:

```
docker build -t contugas_api .
docker run -p 8000:8000 contugas_api
```

Tests unitarios mínimos

```
# tests/test_api.py
import pytest, requests
BASE = "http://127.0.0.1:8000"
```

```
def test_health():
    assert requests.get(BASE+"/health").json()["status"] == "ok"
```

```
def test_predict():
    payload = {"cliente_id": "CLIENTE1", "presion": 11, "temperatura": 24, "volumen": 52}
    r = requests.post(BASE+"/predict", json=payload)
    assert r.status_code == 200 and "anomaly" in r.json()
```

Ejecutar:

```
pytest -q
```

Contacto técnico: correos::::